

The background is a dark blue gradient with a subtle pattern of white dots. On the left side, there are several concentric circles and a large circular scale with degree markings from 140 to 260. Some of the circles have arrows indicating a clockwise direction.

RARITONE BACKEND PROJECT

BACKEND DEVELOPMENT TEAM

NANDINI BHIMINENI

WHAT IS RARITONE?

- Raritone is a virtual fashion platform designed to improve online shopping experiences through personalization and future AI-powered fashion technologies.

Key Features

- Personalized shopping experience
- User profiles
- Product catalog management
- Wishlist and wardrobe management
- User measurements
- Avatar support
- Virtual try-on support
- AI-ready architecture

Project Objective

- To build a secure, scalable, modular, and AI-ready backend system capable of supporting future virtual fashion experiences.



PROBLEM STATEMENT

Existing Challenges

- Users cannot visualize clothing before purchasing
- High product return rates due to fitting issues
- Lack of personalization in fashion shopping
- Limited virtual shopping experiences
- Need for intelligent recommendation systems

Proposed Solution

- Develop a backend system capable of supporting:
 - Virtual try-on
 - Avatar generation
 - Personalized recommendations
 - Measurement-based suggestions
 - AI integration



TECHNOLOGY STACK

- **Backend**
 - Node.js
 - Express.js
- **Database**
 - MongoDB
 - Mongoose
- **Authentication**
 - JWT
 - Google OAuth
- **Cloud**
 - Cloudinary
 - Multer
- **Security**
 - Helmet
 - Rate Limiting
 - Validation



BACKEND ARCHITECTURE

Frontend



REST API



Express Backend



Middleware



Controllers



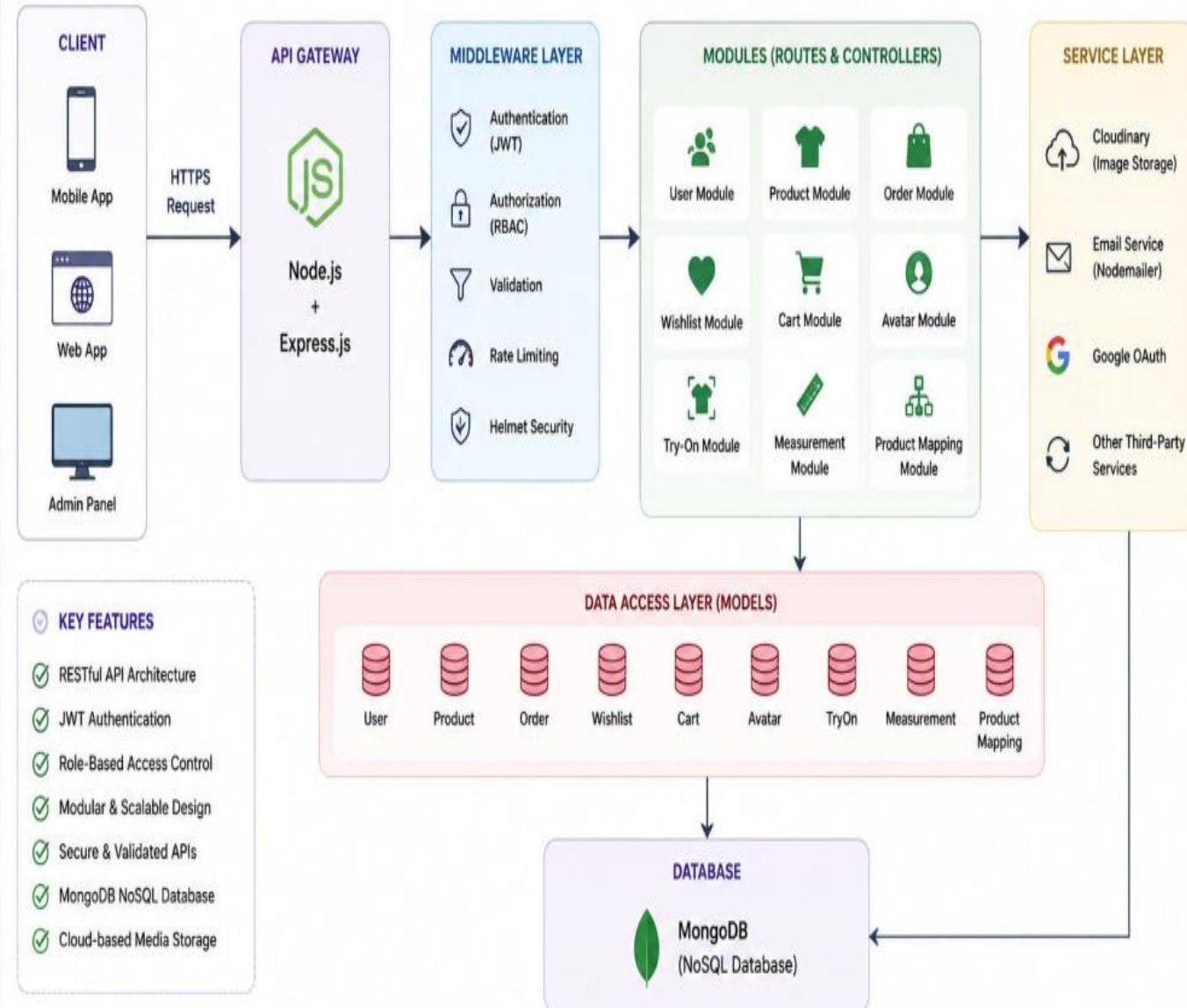
Models



MongoDB + Cloudinary

BACKEND ARCHITECTURE

The Raritone backend is built using Node.js and Express.js with a modular architecture, following RESTful API standards and MongoDB for data storage.



SYSTEM ARCHITECTURE

Frontend
↓
API Layer
↓
Auth Layer
↓
Business Logic
↓
Database Layer
↓
Cloud Storage
↓
AI Layer (Future)



MONGO DB ATLAS

MongoDB Atlas was used as the cloud-based database platform for storing, managing, and accessing application data securely and efficiently. It provides a fully managed database service that simplifies deployment, monitoring, backup, and scaling operations.

- **Usage in the Project**
- Stored user information and activity data
- Managed product and transaction-related records
- Supported data retrieval for analytics and reporting
- Enabled secure and centralized data management
- Provided a scalable foundation for future growth

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

- 127.0.0.1:27017
- Raritone Atlas
 - admin
 - config
 - local
 - oplog.rs
 - replset.election
 - replset.minvalid
 - test
 - avatars
 - carts
 - images
 - measurements
 - orders

test

Raritone Atlas > test

Open MongoDB shell Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total Index size
avatars	-	4.10 kB	0 B	0	0 B	1	4.10 kB
carts	-	4.10 kB	0 B	0	0 B	2	8.19 kB
images	-	4.10 kB	0 B	0	0 B	4	16.38 kB
measurements	-	4.10 kB	0 B	0	0 B	3	12.29 kB
orders	-	4.10 kB	0 B	0	0 B	2	8.19 kB
products	-	36.86 kB	610.00 B	2	305.00 B	3	110.59 kB
profiles	-	4.10 kB	0 B	0	0 B	2	8.19 kB
tryons	-	4.10 kB	0 B	0	0 B	3	12.29 kB
users	-	36.86 kB	264.00 B	1	264.00 B	3	110.59 kB
wardrobes	-	4.10 kB	0 B	0	0 B	4	16.38 kB
wishlists	-	4.10 kB	0 B	0	0 B	4	16.38 kB

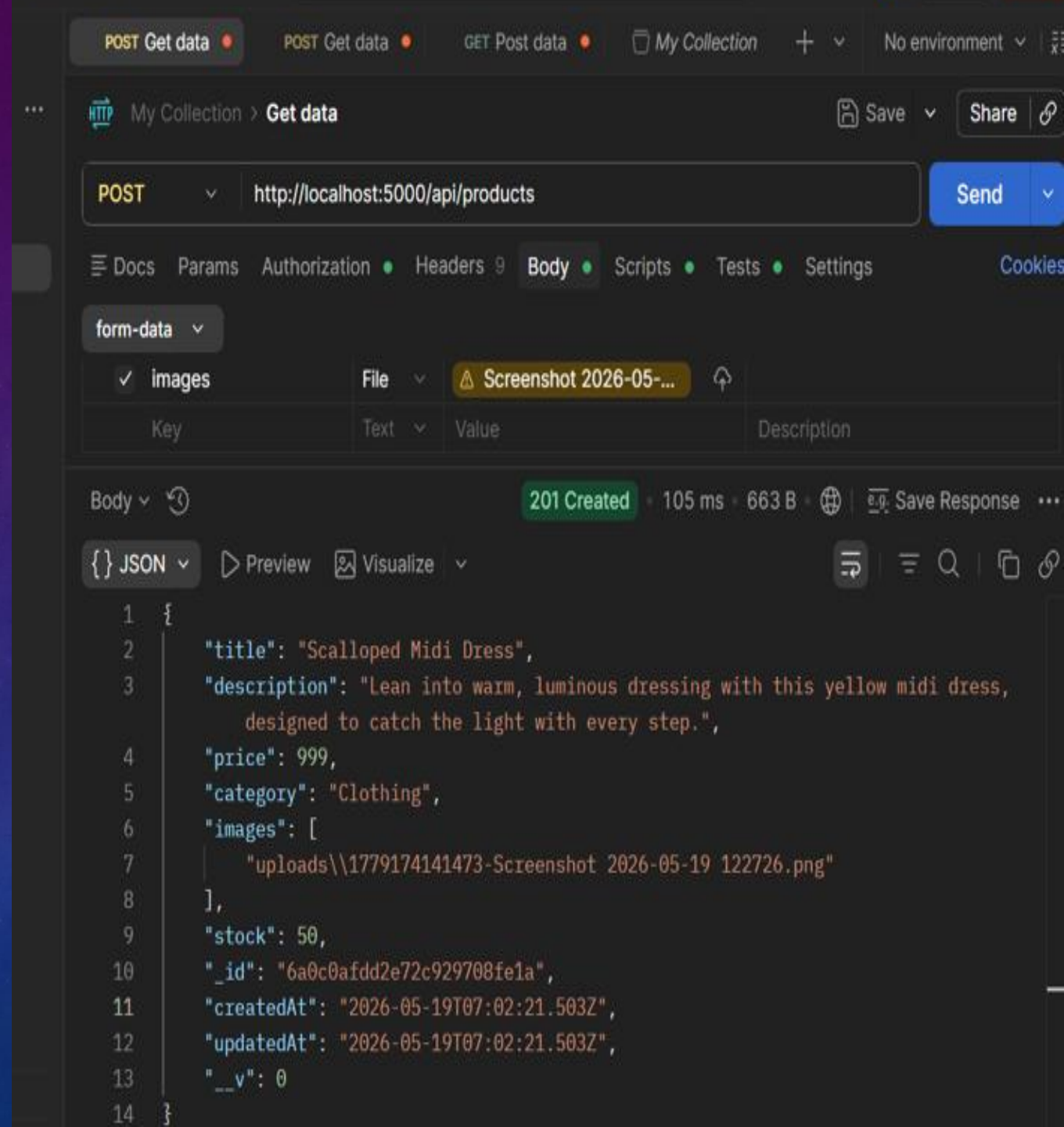
AUTH & PRODUCT APIS

- **Auth APIs**

- POST /api/auth/signup
- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/google

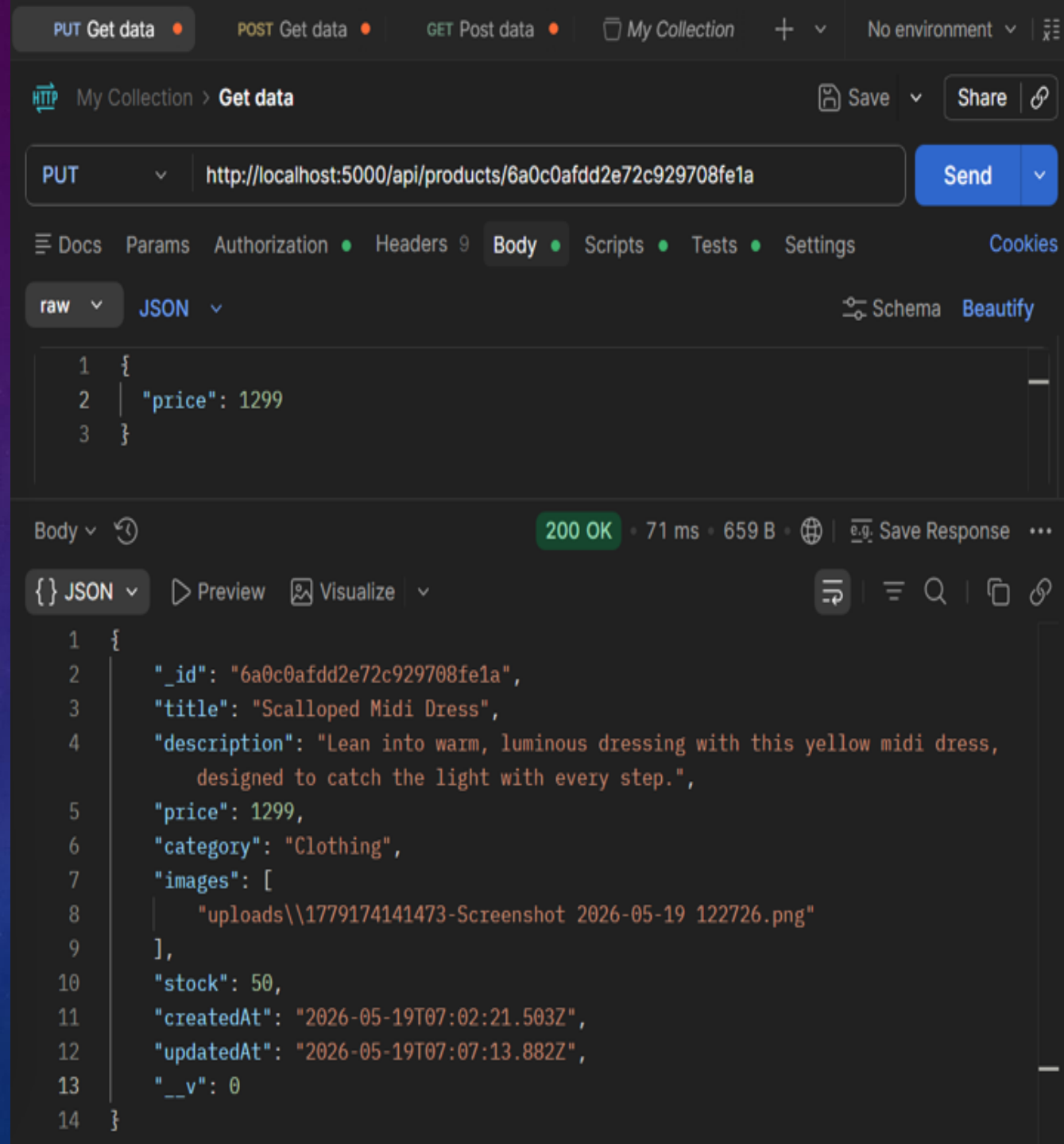
- **Product APIs**

- GET /api/products
- POST /api/products
- PUT /api/products/:id
- DELETE /api/products/:id



USER MANAGEMENT APIS

- **Profile**
 - GET /api/profile
 - PUT /api/profile/update
- **Wishlist**
 - Add / Remove / Get
- **Wardrobe**
 - Add / Remove / Get



SHOPPING APIS

Cart

- Add to cart
- Get cart
- Remove item

Order

- Create order
- Get orders
- Update order

Measurements

- Save / Get / Delete

The screenshot shows a REST client interface with a top navigation bar containing links for Docs, Params, Authorization, Headers (7), Body, Scripts, Tests, and Settings. The Headers tab is active, displaying a table with one header row (Key, Value, Description) and one data row (Authorization, eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp...).

Below the headers, the Body tab is active, showing a 200 OK status with a response time of 23 ms and a size of 318 B. The response is in JSON format, displaying a user object with fields: _id, name, email, and __v.

Key	Value	Descr...	Bulk Edit	Presets
✓ Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp...			

Body ▾ 200 OK • 23 ms • 318 B • Save Response

{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "_id": "6a0aa695f7047d92771b787c",
3   "name": "Nandu",
4   "email": "nandu@gmail.com",
5   "__v": 0
6 }
```


DATABASE DESIGN

- User
- Product
- Profile
- Wishlist
- Wardrobe
- Cart
- Order
- Measurement
- Avatar
- TryOn
- ProductMapping

Raritone-Project-Backend/

```
|
|— config/
|   |— cloudinary.js
|   |— db.js
|
|— controllers/
|   |— authController.js
|   |— avatarController.js
|   |— cartController.js
|   |— googleAuthController.js
|   |— imageController.js
|   |— measurementController.js
|   |— orderController.js
|   |— productController.js
|   |— profileController.js
|   |— tryOnController.js
|   |— wardrobeController.js
|   |— wishlistController.js
|
|— middleware/
|   |— authMiddleware.js
|   |— errorMiddleware.js
|   |— roleMiddleware.js
|   |— uploadMiddleware.js
|   |— validate.js
```

- models/
 - Avatar.js
 - cart.js
 - Image.js
 - measurement.js
 - order.js
 - product.js
 - ProductMapping.js
 - Profile.js
 - TryOn.js
 - user.js
 - Wardrobe.js
 - Wishlist.js
- routes/
 - authRoutes.js
 - avatarRoutes.js
 - cartRoutes.js
 - imageRoutes.js
 - measurementRoutes.js
 - orderRoutes.js
 - productRoutes.js
 - profileRoutes.js
 - tryOnRoutes.js
 - wardrobeRoutes.js
 - wishlistRoutes.js

- uploads/
- utils/
 - cloudinaryUpload.js
 - generateToken.js
- validators/
 - authValidator.js
 - avatarValidator.js
 - loginValidator.js
 - productValidator.js
 - registerValidator.js
 - tryOnValidator.js
- .env
- .gitignore
- app.js
- package.json
- package-lock.json
- README.md
- server.js

DATABASE RELATIONSHIPS

- User → Profile, Cart, Wishlist, Orders, Avatar, Measurements
- Product → Cart, Wishlist, Orders, Mapping
- **Benefits**
 - No redundancy
 - Better performance
 - Easy retrieval

SECURITY

- JWT Authentication
- Refresh Tokens
- Google OAuth
- Role-Based Access Control
- Helmet Security
- Rate Limiting
- Validation Middleware



VIRTUAL TRY-ON FLOW

User selects product



Uploads image/avatar



AI processing



Generated output



Display result

Benefits

- Better visualization
- Reduced returns
- Improved experience

PRODUCT MAPPING

Product relationships

- Recommendation support
- AI personalization

Future Use

- Smart recommendations
- Product matching
- Cross-selling



SCALABILITY

- Redis caching
- Queue systems
- Docker
- Kubernetes
- Load balancing
- MongoDB Atlas

Roadmap

- AI features → Realtime → Caching → Deployment



FUTURE ENHANCEMENTS

- The future development of the Raritone platform aims to enhance user experience, improve personalization, and support large-scale business growth through advanced technologies and analytics.
- **AI-Powered Features**
 - AI-Based Virtual Try-On for realistic product visualization
 - AI Avatar Generation for personalized digital avatars
 - Personalized Product Recommendations based on user preferences and behavior
 - Smart Outfit Suggestions using AI-driven styling insights
 - Fashion Trend Prediction and Preference Analysis

- **Scalability & Performance**

- Redis Caching for faster data retrieval and improved performance
- Docker Containerization for efficient deployment
- Kubernetes for container orchestration and automated scaling
- Load Balancing to distribute traffic and ensure high availability
- Microservices Architecture for better maintainability and scalability

- **Cloud & Security Enhancements**

- AWS S3 Integration for secure and scalable media storage
- Real-Time Notifications for user engagement and order updates
- Multi-Factor Authentication (MFA) for enhanced account security
- Advanced Monitoring and Logging Systems
- Automated Backup and Disaster Recovery Solutions

CONCLUSION

Raritone Backend System is a secure, scalable, and modular virtual fashion platform

- It successfully implements key modules like authentication, products, cart, orders, wishlist, measurements, and avatars
- Built using Node.js, Express.js, and MongoDB with well-structured REST APIs
- Ensures strong security using JWT, RBAC, validation, and rate limiting
- Designed to be AI-ready with future scope for virtual try-on and recommendation systems
- Provides a solid foundation for real-world deployment and future enhancements